Sesión 1 — Redes neuronales desde cero

90 min · notebook `DL1` · transp. 1–25

Objetivo: que salgan sabiendo que una red es *topología + pérdida + forward + backward + regla de actualización*, y que el backward es la regla de la cadena hacia atrás. Todo en numpy: ninguna librería nueva.

Hora	Bloque	Contenido / qué hacer
`0:00`	Apertura	Las tres sesiones y los tres notebooks. Los dos datasets. **Insistir**: aplanar la imagen tira la estructura; eso explicará los resultados de hoy.
`0:08`	Teoría	Red de una capa binaria: combinación lineal → activación → umbral. Por qué no se entrena con el acierto. Pérdidas. **Es regresión logística**: la NLL es la entropía cruzada. Regla de GD: *error* × *entrada*.
`0:20`	**Notebook**	§3.1 completa + ejercicios 1 y 2. Que encuentren un ρ divergente. Que prueben DogsCats.
`0:35`	Puesta en común	Dígitos ≈ 0.95, DogsCats ≈ 0.60. **Pregunta abierta**: ¿por qué? (posición fija vs. objeto en cualquier sitio; un píxel no significa nada por sí solo). Esto motiva capas ocultas y, más adelante, convoluciones.
`0:40`	Teoría	Multiclase: one-hot, softmax, hardmax. El gradiente vuelve a ser $(\boldsymbol{q}-\boldsymbol{y})\boldsymbol{x}^\top$. Estabilidad numérica del softmax (restar el máximo).
`0:48`	**Notebook**	§3.2 + ejercicios 3, 4 y 5. La matriz de confusión del ej. 4 da conversación.
`1:00`	Teoría	Redes multicapa. Grafo forward de la red de 2 capas. Backprop: el error se propaga hacia atrás multiplicado por los pesos y por la derivada de la activación. **Contar el bug**: la versión anterior usaba $(1-h)^2$ en vez de $1-h^2$; entrenaba igual, y por eso hacía falta $\rho/10$ en la capa oculta. De ahí el *gradient checking*.
`1:12`	**Notebook**	§3.3. Primero el gradient check; luego que lo rompan a propósito y vean que la curva de pérdida no se entera. Ejercicio 6.
`1:24`	Cierre	Funciones de activación y saturación. Regresión (solo cambia la última capa y la pérdida). Qué no escala. **Tarea**: los dos tutoriales de PyTorch (30 min).

Ojo con: El entrenamiento de DogsCats con 500 épocas tarda ≈ 100 s; que lo lancen y sigan leyendo mientras. Si alguien va muy rápido, el ejercicio 7 (`MLPClassifier`) le ocupa.

Resultados de referencia (semilla 42, datos normalizados): dígitos binario 1 capa ≈ 0.95 · DogsCats 1 capa ≈ 0.60 · dígitos 10 clases ≈ 0.88 · 2 capas, $n_h = 5$, 2000 épocas, $\rho = 0.5$: dígitos ≈ 0.95 · gradient check $\approx 10^{-9}$.

Sesión 2 — Redes profundas con PyTorch

90 min · notebook `DL2` · transp. 26–41

Objetivo: subir la escalera de abstracción de PyTorch entendiendo qué esconde cada peldaño, y ver las tres formas de controlar el sobreajuste. El modelo es siempre el mismo: lo que cambia es cuánto código escribimos.

Hora	Bloque	Contenido / qué hacer
`0:00`	Apertura	Qué aporta PyTorch. Anunciar el plan: *el mismo modelo escrito cuatro veces*.
`0:05`	Teoría	Tensores (comparten memoria con numpy; `float32` vs `float64`). Autograd: el grafo de cómputo y `backward()`. Los gradientes **se acumulan**: de ahí `zero_grad()`.
`0:15`	**Notebook**	§1.2–1.3 y ejercicio 1. El apartado 1.5 (ejecutar `backward` dos veces) es el que enseña de verdad: preguntarles por qué falla.
`0:25`	Teoría	Los cuatro niveles: (1) tensores + autograd; (2) `nn.Module`/`nn.Parameter`; (3) `nn.Linear` + `nn.CrossEntropyLoss`; (4) `torch.optim`. Señalar que la convergencia más rápida del nivel 3 es por la **inicialización**, no por el modelo.
`0:39`	**Notebook**	§2.1–2.3: ejecutar las cuatro versiones y comprobar que dan lo mismo. Ejercicio 2 (Adam + scheduler).
`0:53`	Teoría	GD vs SGD con minibatches. `DataLoader`. `nn.Sequential`: cuatro capas sin derivar nada (contrastar con las dos transparencias de ayer para derivar dos capas).
`1:01`	**Notebook**	§2.4–2.5. **Pedirles que miren la separación entre la curva de train y la de validación**, no solo el acierto final.
`1:10`	Teoría	Sobreajuste leído en las curvas: acierto que satura vs. pérdida de validación que *crece* (= equivocarse con más confianza). Weight decay, dropout (`train()`/`eval()`), early stopping (mejor modelo, no el último; *paciencia*).
`1:22`	**Notebook**	§3: que lancen los tres experimentos (tardan varios minutos; que arranquen mientras se comenta). Los ejercicios 3–5 quedan como trabajo en casa.

Ojo con: El experimento de weight decay **no mejora el acierto** (0.792 → 0.792) pero baja la pérdida de validación final de 1.08 a 0.79. No lo vendas como una mejora de acierto: el mensaje es que la red regularizada no acierta más, se equivoca con menos confianza. Y el early stopping sobre la *pérdida* selecciona la época ≈139 con acierto 0.760, cuando el mejor acierto (0.792) llega mucho después: **lo que monitorizas es el modelo que te llevas**.

Resultados de referencia: red `Sequential` 4096-200-50-20-10, 300 épocas, SGD $\rho = 0.01$, batch 64 — acierto de validación ≈ 0.79; con dropout 0.5 ≈ 0.84; con weight decay $10^{-2} \approx 0.79$ (pérdida final 0.79 vs 1.08).

Sesión 3 — Redes convolucionales

90 min · notebook `DL3` · transp. 42–57

Objetivo: las dos ideas de una CNN (conectividad local y pesos compartidos), saber calcular tamaños, y ver que en un problema de imagen real se empieza por una red preentrenada.

Hora	Bloque	Contenido / qué hacer
`0:00`	Apertura	Por qué una capa densa es la herramienta equivocada: número de parámetros y falta de invarianza a traslación. Las dos ideas. **Que lancen ya la celda de descarga de CIFAR10**, que tarda.
`0:06`	Teoría	El operador de convolución: kernel, canales, mapas de activación. La fórmula de tamaños y los tres parámetros (`padding`, `stride`, `kernel`). Los parámetros no dependen del tamaño de la imagen.
`0:16`	Teoría	Pooling: max y average. Reduce tamaño, da algo de invarianza, **no tiene parámetros**.
`0:22`	**Notebook**	§2 y ejercicio 1. Que *predigan* el tamaño antes de ejecutar cada caso.
`0:34`	Teoría	Una CNN completa: parte convolucional (extrae) + parte densa (clasifica). Seguir el tamaño a lo largo de la red hasta el $10 \times 8 \times 8 = 640$. El bucle de entrenamiento es el mismo de la sesión 2.
`0:42`	**Notebook**	§3: entrenar 10 épocas (en GPU, ≈ 20 s/época) y mirar el acierto **por clase**. Ejercicio 2 mientras entrena.
`0:57`	Teoría	Data augmentation: transformaciones que no cambian la etiqueta, al vuelo, **solo en train**. Qué transformación es válida es una decisión de modelado (ejemplo de los dígitos y del 6/9).
`1:04`	Teoría	Transfer learning: congelar el backbone, sustituir la cabeza, entrenar solo esa capa. Los números: 11.2M parámetros, 5130 entrenables (0.05 %). Fine-tuning después, con ρ pequeño.
`1:15`	**Notebook**	§4 y §5. Comparar el acierto del transfer learning (sobre 4000 imágenes) con el de la CNN entrenada desde cero sobre las 50000. Cierre del bloque.

Ojo con: Esta sesión **necesita GPU**: Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU. Recordarlo al empezar, no a mitad. La descarga de CIFAR10 (170 MB) y la de los pesos de ResNet18 (45 MB) conviene lanzarlas pronto. El transfer learning usa un subconjunto de 4000/1000 imágenes a propósito: a 224×224 el dataset completo no cabe en el tiempo de clase.

Resultados de referencia: CNN pequeña (7010 parámetros), 10 épocas, Adam $3 \cdot 10^{-4}$ — acierto de test ≈ 0.55–0.60; con augmentation, algo peor al principio y mejor a partir de ≈20 épocas; ResNet18 congelada sobre 4000 imágenes ≈ 0.75–0.80.

Material del bloque

`DL1_NeuralNetworks_numpy.ipynb` · `DL2_DeepNetworks_PyTorch.ipynb` · `DL3_ConvolutionalNetworks.ipynb`
`deeplearning.pdf` (57 transparencias) · `wooclap_deeplearning.md`